

- Vận dụng khung 5V để phân tích một hệ thống Big Data thực tế
- Vận dụng quy trình 3 bước để phân loại một bài toán khai phá dữ liệu
- Cài đặt bằng Python (những kỹ thuật/công cụ vừa học lý thuyết): TF.IDF, mô phỏng Bonferroni, Classification, Clustering, Association Rules, Regression
- Môi trường: dùng lại Python/Jupyter/thư viện đã cài ở Tuần 1 (pandas, scikit-learn)

Chuẩn bị môi trường thực hành

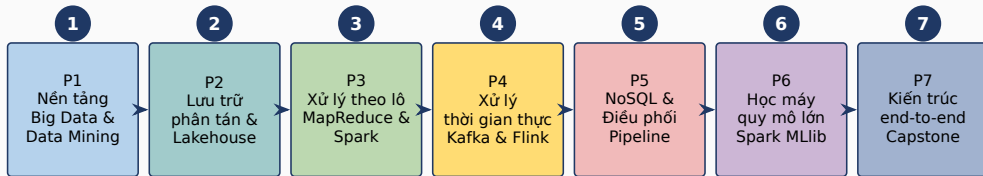
```
# Kích hoạt lại môi trường ảo đã tạo ở Tuần 1
source bigdata-env/bin/activate

# Cài thêm thư viện cần cho buổi thực hành hôm nay
pip install scikit-learn pandas matplotlib mlxtend

# Khởi động Jupyter Notebook
jupyter notebook
```

CASE STUDY: PHÂN TÍCH 5V TRÊN HỆ THỐNG THỰC TẾ

Vận dụng khung 5V đã học vào một hệ thống Big Data cụ thể tại Việt Nam



Bài toán: hệ thống gọi xe công nghệ & giao đồ ăn

- Chọn một hệ thống quen thuộc với sinh viên Việt Nam: nền tảng gọi xe/giao đồ ăn (ví dụ mô hình tương tự Grab, Be, ShopeeFood)
- Hệ thống này xử lý: vị trí GPS tài xế theo thời gian thực, đơn hàng, đánh giá, thanh toán, khuyến mãi cá nhân hóa
- Nhiệm vụ: điền đầy đủ bảng phân tích 5V cho hệ thống này, có số liệu minh họa cụ thể

Dữ liệu mô tả hệ thống (giả định minh họa)

Chỉ số	Giá trị ước lượng
Số chuyến/đơn hàng mỗi ngày (toàn quốc)	Hàng triệu
Tần suất cập nhật vị trí GPS	Mỗi 2–5 giây/tài xế
Số loại dữ liệu	Vị trí, văn bản (đánh giá), ảnh (món ăn), giao dịch
Yêu cầu độ trễ ghép tài xế–khách	Dưới 1–2 giây

- Volume
 - Ước lượng: hàng triệu chuyến/đơn mỗi ngày $\times$ nhiều bản ghi vị trí GPS mỗi chuyến $\Rightarrow$ hàng tỷ bản ghi/ngày
 - Câu hỏi: dữ liệu này có thể lưu và truy vấn hiệu quả trên một CSDL quan hệ đơn lẻ không?
- Velocity
 - Ghép tài xế–khách hàng phải xảy ra trong **vài giây** — đòi hỏi xử lý thời gian thực
 - Vị trí GPS là luồng dữ liệu **liên tục, vô hạn** — liên hệ trực tiếp “dữ liệu streaming” đã học ở khung phân loại bài toán

- Variety
 - Vị trí GPS (dữ liệu có cấu trúc, dạng chuỗi thời gian)
 - Đánh giá/bình luận của khách (văn bản phi cấu trúc)
 - Ảnh món ăn, giấy tờ tài xế (dữ liệu hình ảnh)
 - Giao dịch thanh toán (dữ liệu có cấu trúc, cần độ chính xác cao)
- Veracity
 - GPS có thể lệch do nhiễu tín hiệu ở khu vực nhà cao tầng
 - Đánh giá có thể bị thao túng (đánh giá ảo, mua sao)
 - Hệ quả: mô hình ghép tài xế/gợi ý cần cơ chế lọc nhiễu và phát hiện gian lận

- Value không nằm ở dữ liệu thô, mà ở **quyết định** được tạo ra từ dữ liệu:
- Ghép tài xế tối ưu → giảm thời gian chờ, tăng doanh thu
- Định giá động (surge pricing) dựa trên cung–cầu thời gian thực
- Gợi ý món ăn cá nhân hóa dựa trên lịch sử đơn hàng (liên hệ Association Rules & Clustering)
- Phát hiện gian lận khuyến mãi/đánh giá ảo (liên hệ Classification)

Bài tập nhóm: tự chọn một hệ thống khác

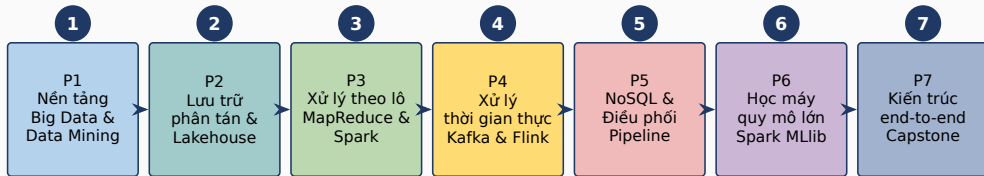
- Làm việc theo nhóm 3–4 người, chọn **một hệ thống Big Data khác** (gợi ý: mạng xã hội học tập, hệ thống thư viện số, cảm biến IoT nông nghiệp, hệ thống ngân hàng số)
- Điền bảng 5V cho hệ thống đã chọn:
 - Volume: quy mô dữ liệu ước lượng là bao nhiêu?
 - Velocity: có cần xử lý thời gian thực không? Độ trễ yêu cầu?
 - Variety: có bao nhiêu loại/định dạng dữ liệu?
 - Veracity: rủi ro về chất lượng dữ liệu là gì?
 - Value: quyết định/giá trị cụ thể nào được tạo ra?
- Thời gian: 15 phút thảo luận nhóm

Tiêu chí đánh giá & thảo luận lớp

- Mỗi nhóm cử đại diện trình bày trong 2 phút
- Tiêu chí: có **số liệu/ví dụ cụ thể** cho từng V (không chỉ định nghĩa chung chung)
- Câu hỏi thảo luận mở: hệ thống nào trong lớp có Veracity **thấp nhất**? Vì sao? Cách khắc phục là gì?

BÀI TẬP: NHẬN DIỆN & PHÂN LOẠI BÀI TOÁN KHAI PHÁ DỮ LIỆU

Áp dụng quy trình 3 bước đã học ở phần lý thuyết



Nhắc lại quy trình 3 bước

- **Bước 1:** Xác định kỹ thuật khai phá dữ liệu (Classification / Clustering / Association Rules / Regression)
- **Bước 2:** Xác định đặc điểm dữ liệu (có nhãn? dạng đồ thị? luồng vô hạn?)
- **Bước 3:** Chọn mô hình tính toán phù hợp (single-machine / MapReduce / streams)

Ví dụ mẫu đã giải: “Gợi ý bạn bè trên mạng xã hội”

- **Bước 1:** Đây là bài toán tìm các cặp người dùng “tương tự/gần nhau” trong đồ thị quan hệ → gần với **Clustering / Similar Items**, không phải Classification thuần túy
- **Bước 2:** Dữ liệu dạng **đồ thị** (graph) — người dùng là đỉnh, quan hệ bạn bè là cạnh; không có nhãn “đúng/sai” rõ ràng cho mỗi gợi ý
- **Bước 3:** Đồ thị mạng xã hội thường **quá lớn** để vừa RAM một máy → cần MapReduce/xử lý đồ thị phân tán (sẽ học kỹ thuật cụ thể ở các phần sau)

Đề bài: phân loại 8 bài toán sau

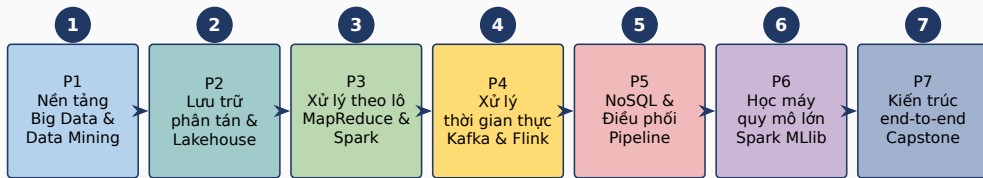
#	Bài toán
1	Dự đoán một bệnh nhân có mắc tiểu đường hay không, dựa trên hồ sơ bệnh án đã có nhãn
2	Đếm số lượng từ khóa duy nhất xuất hiện trong luồng tìm kiếm của một công cụ search 24/7
3	Nhóm các bài báo tin tức thành các chủ đề mà không biết trước chủ đề nào
4	Tìm các mặt hàng thường được mua chung trong siêu thị
5	Dự đoán giá cổ phiếu ngày mai dựa trên dữ liệu lịch sử
6	Phát hiện giao dịch gian lận ngay khi giao dịch thẻ vừa xảy ra
7	Tìm cộng đồng (nhóm bạn bè gắn kết chặt) trong mạng xã hội hàng trăm triệu người dùng
8	Dự đoán số sao đánh giá (1–5 sao) một khách hàng sẽ cho một bộ phim

Đáp án gợi ý (thảo luận cùng giảng viên)

#	Kỹ thuật	Đặc điểm dữ liệu	Mô hình tính toán
1	Classification	Có nhãn	Single-machine / MapReduce
2	(Đếm phần tử duy nhất)	Luồng vô hạn	Streams (xác suất)
3	Clustering	Không nhãn, văn bản	MapReduce (nếu lớn)
4	Association Rules	Dạng giỏ hàng	MapReduce (A-Priori)
5	Regression	Có nhãn (giá thực tế)	Single-machine / streams
6	Classification	Có nhãn, luồng liên tục	Streams / online
7	Clustering (đồ thị)	Đồ thị, khối lượng lớn	MapReduce / đồ thị phân tán
8	Regression / Classification	Có nhãn	Single-machine / MapReduce

MINI-LAB: CÀI ĐẶT BẰNG PYTHON

Sáu bài lab ngắn — mỗi bài gắn trực tiếp với một nội dung lý thuyết vừa học



Lab 1 — Tính TF.IDF bằng Python

```
from sklearn.feature_extraction.text import TfidfVectorizer

docs = [
    "big data can duoc xu ly phan tan",
    "spark xu ly du lieu lon theo lo",
    "kafka xu ly du lieu thoi gian thuc",
]

vectorizer = TfidfVectorizer()
X = vectorizer.fit_transform(docs)

import pandas as pd
df = pd.DataFrame(X.toarray(), columns=vectorizer.get_feature_names_out())
print(df.round(3))
```

Lab 1 — kết quả & giải thích

- Kết quả là một ma trận: mỗi hàng là một văn bản, mỗi cột là một từ, giá trị là điểm TF.IDF
- Từ xuất hiện ở **cả 3 văn bản** (ví dụ “xu_ly”, “du_lieu”) sẽ có điểm TF.IDF **thấp** — đúng như công thức đã học (*IDF* nhỏ khi n_i lớn)
- Từ chỉ xuất hiện ở **một** văn bản (ví dụ “kafka”, “spark”) sẽ có điểm TF.IDF **cao** — đây là từ đặc trưng cho văn bản đó
- So sánh trực tiếp với ví dụ số đã tính tay ở phần lý thuyết

Lab 2 — Mô phỏng nguyên lý Bonferroni bằng Python

```
import random
from collections import defaultdict

N_PEOPLE, N_HOTELS, N_DAYS, P_VISIT = 2000, 50, 30, 0.3 # thu nhỏ tu  $10^9$ 

by_hotel_day = defaultdict(list) # (hotel, day) -> [nguoi]
for day in range(N_DAYS):
    for person in range(N_PEOPLE):
        if random.random() < P_VISIT:
            hotel = random.randrange(N_HOTELS)
            by_hotel_day[(hotel, day)].append(person)

pair_days = defaultdict(set) # cap nguoi -> tap ngay da gap nhau
for (hotel, day), people in by_hotel_day.items():
    for i in range(len(people)):
        for j in range(i + 1, len(people)):
            pair_days[tuple(sorted((people[i], people[j])))].add(day)

suspects = sum(1 for days in pair_days.values() if len(days) >= 2)
print(f"So cap 'kha nghi' (du lieu hoan toan ngau nhien): {suspects}")
```

- Với các tham số thu nhỏ trong code (2.000 người, 50 khách sạn, 30 ngày), chương trình vẫn tìm ra một số lượng **đáng kể** các cặp “khả nghi” — dù dữ liệu hoàn toàn ngẫu nhiên (được sinh bằng random)
- Thử tăng N_PEOPLE hoặc N_DAYS và quan sát: số cặp khả nghi tăng **nhANH hơn nhiều** so với tốc độ tăng dữ liệu — đúng như công thức tổ hợp đã học
- Đây chính là bằng chứng thực nghiệm cho nguyên lý Bonferroni: **không có bất kỳ “kẻ xấu” nào** trong dữ liệu mô phỏng này, nhưng thuật toán vẫn tìm ra rất nhiều “ngghi phạm giả”

Lab 3 — Clustering với K-Means

```
from sklearn.cluster import KMeans
import numpy as np

# du lieu gia dinh: [so don hang/thang, gia tri don trung binh]
khach_hang = np.array([
    [2, 150], [3, 180], [1, 120],      # nhom mua it, gia tri thap
    [15, 90], [18, 100], [20, 85],     # nhom mua nhieu, gia tri thap (an vat)
    [4, 900], [5, 1200], [3, 1000],    # nhom mua it, gia tri cao (VIP)
])

model = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = model.fit_predict(khach_hang)

print("Nhan cum:", labels)
print("Tam cum:\n", model.cluster_centers_.round(1))
```

Lab 3 — trực quan hóa & giải thích kết quả

- K-Means chia 9 khách hàng thành 3 cụm dựa trên (số đơn hàng/tháng, giá trị đơn trung bình) — **không cần nhãn trước**, đúng bản chất học không giám sát
- Kết quả kỳ vọng: cụm “mua ít, giá trị thấp”, cụm “mua nhiều, giá trị thấp” (khách mua đồ ăn vặt), và cụm “mua ít nhưng giá trị cao” (khách VIP)
- Liên hệ trực tiếp: đây chính là kỹ thuật **phân khúc khách hàng** đã nêu trong phần lý thuyết Clustering
- `model.cluster_centers_` chính là các **centroid** — bản tóm tắt của mỗi cụm

Lab 4 — Association Rules (Market-Basket) thử công

```
gio_hang = [  
    {"Bim", "Bia", "Sua"},  
    {"Bim", "Bia"},  
    {"Bim", "Bia", "Banh"},  
    {"Sua", "Banh"},  
]  
  
def support(itemset, baskets):  
    count = sum(1 for b in baskets if itemset.issubset(b))  
    return count / len(baskets)  
  
def confidence(A, B, baskets):  
    return support(A | B, baskets) / support(A, baskets)  
  
A = {"Bim"}; B = {"Bia"}  
print(f"support(Bim => Bia)      = {support(A|B, gio_hang):.2f}")  
print(f"confidence(Bim => Bia) = {confidence(A, B, gio_hang):.2f}")
```

Lab 4 — kết quả & giải thích

- Kết quả: $\text{support} = 0.75$, $\text{confidence} = 1.00$ — **khớp chính xác** với ví dụ số đã tính tay ở phần lý thuyết
- Thử thêm luật khác, ví dụ **Bánh** $\Rightarrow$ **Sữa**, và so sánh support/confidence — luật nào “mạnh” hơn?
- Với dữ liệu thật (hàng triệu giỏ hàng), cách tính “duyet toàn bộ” này không khả thi — đây là lý do cần thuật toán A-Priori (đã giới thiệu ở lý thuyết) để cắt tỉa không gian tìm kiếm

Lab 5 — Classification với Decision Tree

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
import numpy as np

# du lieu gia dinh: [so gio hoc/tuan, ti le diem danh] -> dau (1) / rot (0)
X = np.array([[2,60],[3,70],[8,95],[1,50],[9,98],[4,75],[7,90],[2,55]])
y = np.array([0,0,1,0,1,0,1,0])

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=1)
clf = DecisionTreeClassifier(max_depth=3, random_state=1)
clf.fit(X_train, y_train)

print("Do chinh xac tren tap test:", clf.score(X_test, y_test))
print("Du doan cho [6 gio hoc, 85% diem danh]:", clf.predict([[6, 85]]))
```

- Mô hình học từ dữ liệu **đã có nhãn** (đậu/rót) — đúng bản chất **Classification** có giám sát
- `clf.score(...)` cho biết độ chính xác trên tập kiểm tra — một chỉ số quan trọng để đánh giá mô hình trước khi triển khai
- Decision Tree dễ diễn giải: có thể trực quan hóa thành các luật “nếu số giờ học $> X$ thì...” — đúng như đã mô tả ở phần lý thuyết
- Ở quy mô lớn (hàng triệu sinh viên/hồ sơ), Spark MLlib cung cấp phiên bản Decision Tree chạy phân tán — sẽ thực hành ở Phần 6

Lab 6 — Regression với Linear Regression

```
from sklearn.linear_model import LinearRegression
import numpy as np

# du lieu gia dinh: [chi phi quang cao (trieu VND)] -> doanh thu (trieu VND)
X = np.array([[10],[20],[30],[40],[50],[60]])
y = np.array([120, 210, 300, 405, 500, 590])

model = LinearRegression()
model.fit(X, y)

print(f"beta_0 (intercept) = {model.intercept_:.2f}")
print(f"beta_1 (he so)      = {model.coef_[0]:.2f}")
print("Du doan doanh thu khi chi 45 trieu quang cao:", model.predict([[45]]))
```

- `model.coef_` và `model.intercept_` chính là β_1 và β_0 trong công thức $y = \beta_0 + \beta_1 x$ đã học ở lý thuyết
- Mô hình được huấn luyện bằng phương pháp **least squares** — tối thiểu hóa tổng bình phương sai số
- Đây là ví dụ đơn giản nhất của Regression; ở Phần 6, Spark MLlib sẽ huấn luyện hồi quy trên hàng triệu bản ghi bằng **Stochastic Gradient Descent**

Tổng hợp 6 lab vừa thực hiện

Lab	Kỹ thuật/khái niệm	Liên hệ lý thuyết
1	TF.IDF	Mục “Kiến thức nền tảng”
2	Mô phỏng Bonferroni	Mục “Giới hạn thống kê”
3	Clustering (K-Means)	Kỹ thuật khai phá dữ liệu #2
4	Association Rules	Kỹ thuật khai phá dữ liệu #3
5	Classification (Decision Tree)	Kỹ thuật khai phá dữ liệu #1
6	Regression (Linear Regression)	Kỹ thuật khai phá dữ liệu #4

- Thử áp dụng Lab 3–6 trên một bộ dữ liệu thật nhỏ tải từ Kaggle hoặc UCI Machine Learning Repository
- Viết lại Lab 2 (mô phỏng Bonferroni) với tham số gần với ví dụ gốc hơn (10^9 người) — ước lượng thời gian chạy sẽ thay đổi thế nào, và vì sao cần MapReduce cho quy mô thật?
- Hoàn thiện bảng 5V cho hệ thống nhóm đã chọn, chuẩn bị nộp báo cáo ngắn (1 trang)

Checklist hoàn thành buổi thực hành

- ☐ Đã điền đầy đủ bảng 5V cho case study gọi xe/giao đồ ăn
- ☐ Đã hoàn thành bảng 5V cho hệ thống tự chọn theo nhóm
- ☐ Đã phân loại đúng ít nhất 6/8 bài toán trong bài tập phân loại
- ☐ Đã chạy thành công cả 6 mini-lab và đối chiếu kết quả với lý thuyết
- ☐ Đã ghi chú lại các câu hỏi còn thắc mắc để hỏi ở đầu Tuần 3

Chuẩn bị cho Tuần 3 — Phần 2: Lưu trữ phân tán hiện đại

- Đọc trước: White — *Hadoop: The Definitive Guide* (chương giới thiệu HDFS); tài liệu Delta Lake/Iceberg đã tải về trong `TaiLieuThamKhao/P2_LuuTruPhanTan`
- Nội dung sắp học: kiến trúc HDFS, Cloud Object Storage, định dạng cột (Parquet/ORC), kiến trúc Lakehouse
- Thực hành sắp tới: thao tác lệnh HDFS cơ bản, đọc/ghi Parquet bằng Spark, tạo bảng Delta Lake

Hỏi & Đáp



Hẹn gặp lại ở Tuần 3 — Phần 2: Lưu trữ phân tán hiện đại

Tài liệu tham khảo đầy đủ của Phần 1 được liệt kê ở slide tổng kết lý thuyết